

EDUGENIE

Google Gemini Powered Learning Assistant

PROJECT DOCUMENTATION | IMPLEMENTATION & FUNCTIONAL VALIDATION

A lightweight, modular AI learning companion for explanations, Q&A, quizzes, summaries, and personalized learning paths.

05	02	05	01
Core learning capabilities	AI model strategies	REST endpoints	Web interface

Submitted by

Varshini Loviya C ,Sakthi Harish M, Pranesh M, Sri Karthick Kumar
G

Mentor: Krishnan N • Date: 23/09/2026

01 | PROJECT OVERVIEW

EduGenie at a Glance

A concise view of the problem, solution, user journeys, and implementation approach.

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through generative AI. The system supports students and self-learners by turning questions, passages, and topics into concise answers, explanations, quizzes, summaries, and structured learning recommendations.

Q&A Ask and answer	EXPLAIN Simplify concepts	QUIZ Generate MCQs	SUMMARY Condense content	LEARN Build a path
----------------------------------	-------------------------------------	------------------------------	------------------------------------	------------------------------

What the project solves

- Reduces the effort required to understand complex academic concepts.
- Provides multiple learning modes from a single web interface.
- Combines cloud-based generative AI with a lightweight local explanation model.
- Organizes learning recommendations from beginner to advanced levels.
- Keeps the backend modular so each capability can evolve independently.

Representative user scenarios

M1	Question discovery A learner asks: "Which is the largest ocean?" and receives a concise answer.
M2	Knowledge check A learner selects a topic such as the Pythagoras Theorem and generates a quiz to test understanding.
M3	Structured learning A learner exploring SQL requests a path containing beginner-to-advanced topics, timelines, and learning resources.

02 | SOLUTION ARCHITECTURE

From learner input to learning output

The architecture separates presentation, API routing, and specialized AI modules.

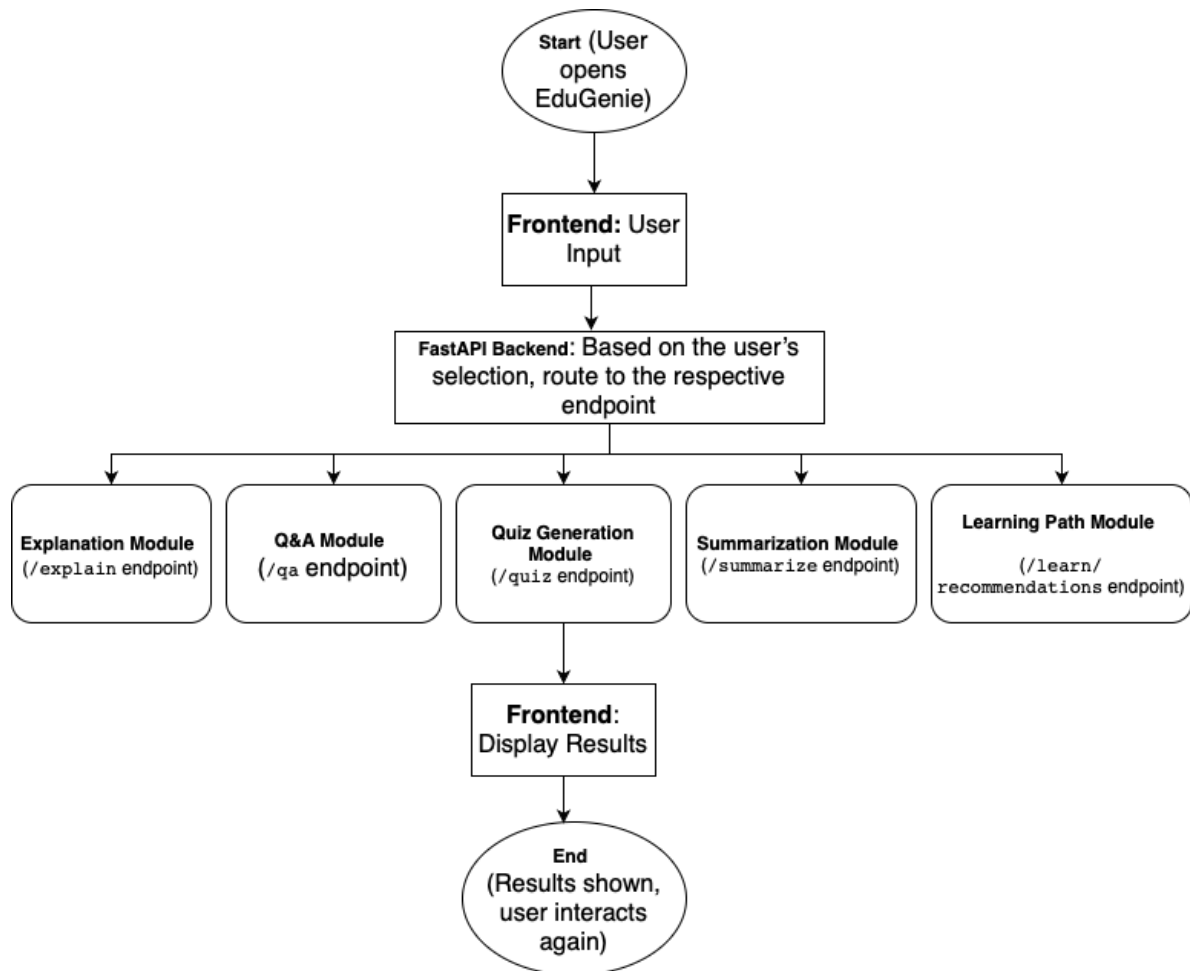


Figure 1. EduGenie request flow: user → frontend → FastAPI → specialized modules → results.

Architecture principle

The FastAPI backend acts as the routing layer. Each endpoint connects the incoming request to the module responsible for the requested learning task.

Flow explained

01

User interaction The learner opens EduGenie and provides a question, topic, or passage.

02

Frontend capture The HTML interface collects the input and the selected learning task.

03

FastAPI routing The backend routes the request to the relevant REST endpoint.

04	Specialized processing Q&A, explanation, quiz, summary, or learning-path logic processes the request.
05	Result presentation The generated response is returned to the web interface for continued interaction.

03 | TECHNOLOGY FOUNDATION

Technology stack & prerequisites

The original implementation uses a lightweight Python web stack with Google Gemini integration.

Core technologies

Technology	Role in EduGenie	Project usage
Python 3.10+	Application language	Backend implementation
FastAPI	REST API framework	Routes learning requests
Google Gemini API	Cloud generative AI	Q&A, quizzes, summaries, learning paths
LaMini-Flan-T5-783M	Lightweight local model	Concept explanations
HTML + CSS	Presentation layer	Web interface
Uvicorn	ASGI server	Local application serving
Jinja2	HTML templating	Template rendering

Prerequisite setup

- Install Python 3.10+ and verify the installation using `python --version`.
- Set up the FastAPI framework and supporting Python dependencies.
- Create and securely store a Google Gemini API key.
- Install Uvicorn for running the FastAPI application.
- Use Jinja2 for HTML template rendering.

Security note

The project documentation specifies securely storing the Gemini API key. Credentials should not be embedded directly in source code or shared in the project submission.

04 | PROJECT STRUCTURE

Modular project organization

The folder structure mirrors the learning capabilities, making the system easy to understand and extend.

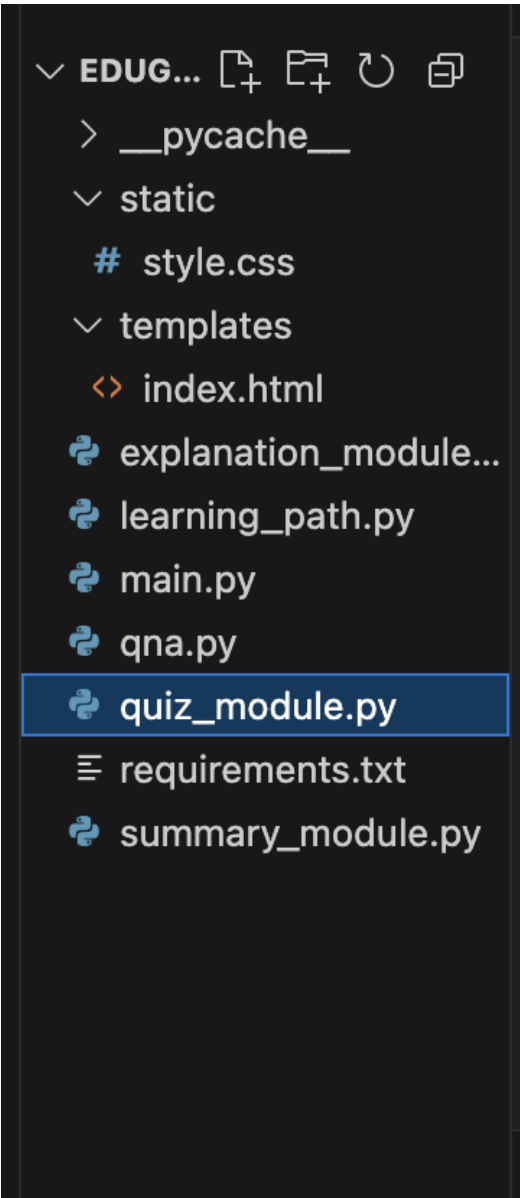


Figure 2. Project folder structure used by EduGenie.

Application modules

File / folder	Responsibility
main.py	FastAPI application and endpoint definitions
explanation_module.py	Concept explanation logic
qna.py	Question-answering logic
quiz_module.py	Quiz generation

summary_module.py	Text summarization
learning_path.py	Personalized learning recommendations
templates/index.html	HTML frontend
static/style.css	Interface styling
requirements.txt	Python dependency list

Design advantage
Separating the capabilities into focused modules keeps the application readable and supports incremental enhancement without redesigning the complete system.

Two-model approach

EduGenie combines cloud reasoning with a lightweight local explanation model.

GEMINI 1.5 PRO

Used for Q&A, summarization, quiz generation, and learning paths. The document highlights advanced reasoning, structured outputs, and cloud inference.

LAMINI-FLAN-T5-783M

Used for concept explanation. The document describes it as instruction-tuned, lightweight, and CPU-compatible.

Why the split matters

The implementation does not force every task through one model. Instead, the architecture assigns cloud-based generative tasks to Gemini while keeping concept explanation on a smaller local model. This supports the project’s goal of combining cloud intelligence with a low-footprint application.

Implementation milestone

Milestone 1 establishes the model selection and architecture before the functional modules are implemented.

06 | CORE FUNCTIONALITIES

The five learning modes

Each capability has a dedicated module and a clear learner-facing outcome.

1. Concept Explanation

The Explanation Module uses LaMini-Flan-T5-783M to convert a topic into a simple, readable explanation. The project emphasizes clarity and brevity so that beginners and self-learners can understand foundational concepts without unnecessary complexity.

```

Explanation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
  
```

Figure 3. Explanation module implementation.

2. Question & Answer

The QnA Module uses Gemini 1.5 Pro to answer general knowledge and academic questions. Its role is to provide contextual, AI-driven assistance across educational topics.

```

def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in QnA: {e}"
  
```

Figure 4. Q&A module implementation.

06 | CORE FUNCTIONALITIES

Assessment & revision

The second group of modules transforms learning material into practice and quick-review formats.

3. Quiz Generation

The Quiz Module generates three multiple-choice questions from a supplied passage. Each question contains four options and a correct answer. The implementation requests structured JSON, removes Markdown code fences when necessary, and parses the cleaned response into a Python list.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧹 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
    except Exception as e:
        return f"⚠️ Error in Summary: {e}"
```

Figure 5. Quiz-generation logic and structured output approach.

4. Summarization

The Summary Module uses Gemini to turn longer passages into concise, easy-to-understand text while retaining the core information and reducing redundancy.

```
def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in Summary: {e}"
```

Figure 6. Summary module implementation.

06 | CORE FUNCTIONALITIES

Personalized learning path

The learning-path module extends EduGenie from answering questions to guiding a learner through a topic.

The `get_learning_recommendations` function uses Google Gemini to generate a structured path for a selected topic. The prompt requests beginner-to-advanced concepts, difficulty organization, estimated progression, and learning resources such as videos, articles, or books.

```
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (1
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🗨️ Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc() # ✅ This gives full error trace
    return f"❌ Error occurred: {str(e)}"
```

Figure 7. Learning-path generation logic.

Learner outcome

The feature is designed to provide stepwise guidance rather than a single answer, helping a learner move from foundational topics toward more advanced concepts.

REST endpoint layer

FastAPI connects the user interface with the five learning capabilities.

Endpoint	Purpose
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Text summarization
/learn/recommendations	Learning recommendations

```
main.py
40
41 # Q&A - GET API using Gemini
42 @app.get("/qa")
43 async def answer_question(question: str = Query(...)):
44     answer = answer_question_with_gemini(question)
45     return {"answer": answer}
46
47 # Explanation - POST API
48 @app.post("/explain/")
49 async def explain_api(request: Request):
50     data = await request.json()
51     topic = data.get("topic")
52     if not topic:
53         return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
54     explanation = explain_topic(topic)
55     return {"topic": topic, "explanation": explanation}
56
57 # Summarization - POST API
58 @app.post("/summarize/")
59 async def summarize_api(request: Request):
60     data = await request.json()
61     text = data.get("text")
62     if not text:
63         return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
64     summary = summarize_text(text)
65     return {"summary": summary}
66
```

API routes — primary endpoint logic

```

67 # Quiz Generation - POST
68 @app.post("/quiz")
69 async def quiz_api(request: Request):
70     data = await request.json()
71     text = data.get("text")
72     if not text:
73         return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
74     quiz = generate_quiz(text)
75     print("Generated quiz:", quiz) # DEBUG
76     return JSONResponse(content={"quiz": quiz})
77
78 # Learning Recommendations - GET
79 @app.get("/learn/recommendations")
80 async def learning_recommendation_api(topic: str = Query(...)):
81     recommendation = get_learning_recommendations(topic)
82     return {"topic": topic, "recommendation": recommendation}
83
```

API routes — quiz and learning-path handling

API design

Each endpoint is connected to a corresponding module, keeping routing logic separate from the actual AI-processing logic.

A focused learner interface

The frontend keeps the interaction simple: select a task, provide input, submit, and view the result.

```

1 templates > <> index.html > ...
2     <html lang="en">
3     <head>
86     </head>
87     <body>
88         <h1>Welcome to EduGenie 🤖</h1>
89         <p>Your personal AI tutor for learning support!</p>
90
91         <!-- Q&A -->
92         <form id="qaForm">
93             <label><strong>Ask EduGenie a Question:</strong></label><br><br>
94             <input type="text" id="question" placeholder="Why is the sky blue?" required>
95             <button type="submit">Get Answer</button>
96         </form>
97         <div id="qaResult" class="output"></div>
98
99         <!-- Explanation -->
100        <form id="explainForm">
101            <label><strong>Need an Explanation?</strong></label><br><br>
102            <input type="text" id="topic" placeholder="Photosynthesis" required>
103            <button type="submit">Explain</button>
104        </form>
105        <div id="explanationResult" class="output"></div>
106
107        <!-- Summary -->
108        <form id="summaryForm">
109            <label><strong>Summarize a Paragraph:</strong></label><br><br>
110            <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
111            <button type="submit">Summarize</button>
112        </form>
113        <div id="summaryResult" class="output"></div>
114
115        <!-- Quiz -->
116        <form id="quizForm">
117            <label><strong>Generate a Quiz:</strong></label><br><br>
118            <input type="text" id="quizText" placeholder="Solar System" required>
119            <button type="submit">Generate Quiz</button>
120        </form>
121        <div id="quizResult" class="output"></div>

```

Figure 8. HTML frontend integration.

Interface elements

- Task selection for Explain, QnA, Quiz, Summary, and Recommend Path.
- Text input area for questions, topics, or passages.
- Dedicated action buttons for each learning task.
- Result containers that display responses below the input.
- Responsive CSS styling for a simple web experience.

Live integration

Form submission sends requests to the FastAPI backend, and the generated result is returned to the web interface for the learner to continue interacting with EduGenie.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python3.11 + - [ ] ... ^ X
```

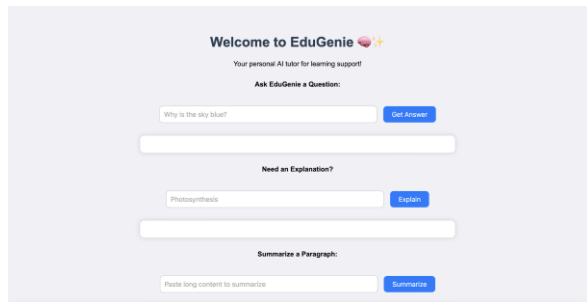
```
(base) telladivvyreddy@telas-MacBook-Air edugenie % uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/Users/telladivvyreddy/Desktop/edugenie']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reload process [17749] using StarReload
/opt/anaconda3/lib/python3.11/site-packages/torch/utils/_pytree.py:185: FutureWarning: optree is installed but the version is too old to support PyTorch Dynamo in C++ pytree. C++ pytree support is disabled. Please consider upgrading optree using 'python3 -m pip install --upgrade 'optree>=0.13.0''.
warnings.warn(
INFO: Started server process [17751]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 9. Local Uvicorn run showing the application starting successfully.

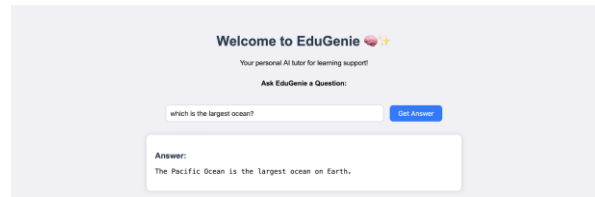
09 | FUNCTIONAL VALIDATION

What was demonstrated

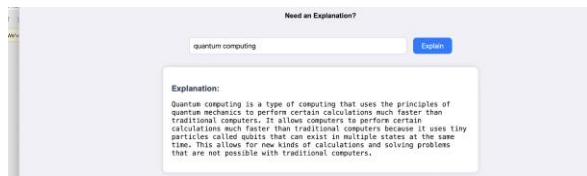
The original submission includes screenshots showing the major user-facing capabilities in operation.



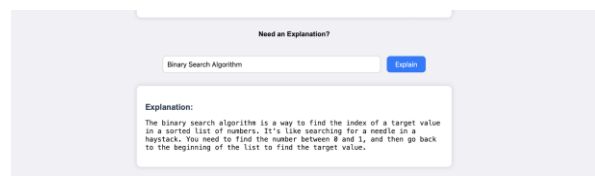
EduGenie interface



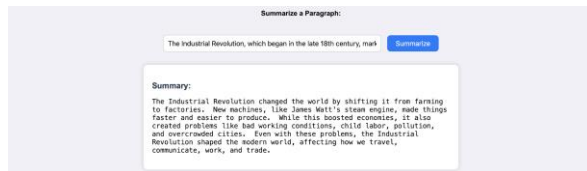
Q&A response



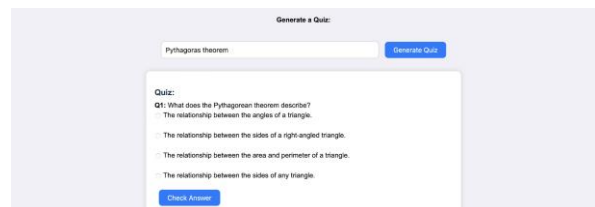
Topic explanation



Summary output



Quiz generation



Quiz interaction

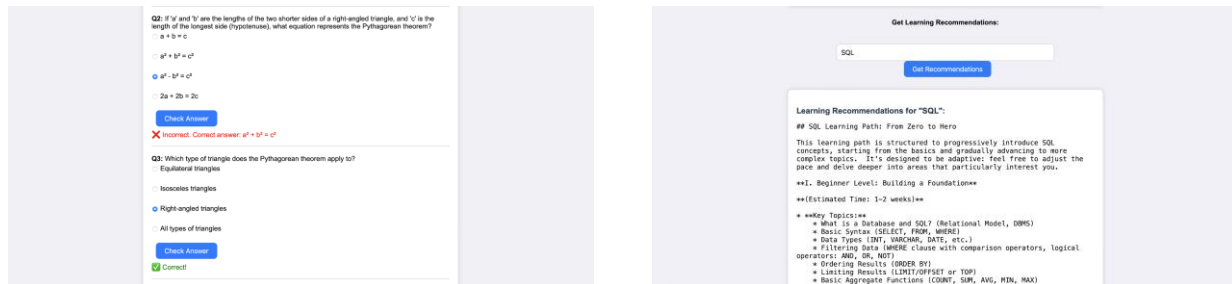
Validation coverage

- Ask a question and display the generated answer.
- Request an explanation for a selected topic.
- Summarize a long paragraph.
- Generate a quiz containing multiple questions and answer options.
- Provide learning recommendations from beginner to advanced levels.

10 | LEARNING RECOMMENDATION DEMONSTRATION

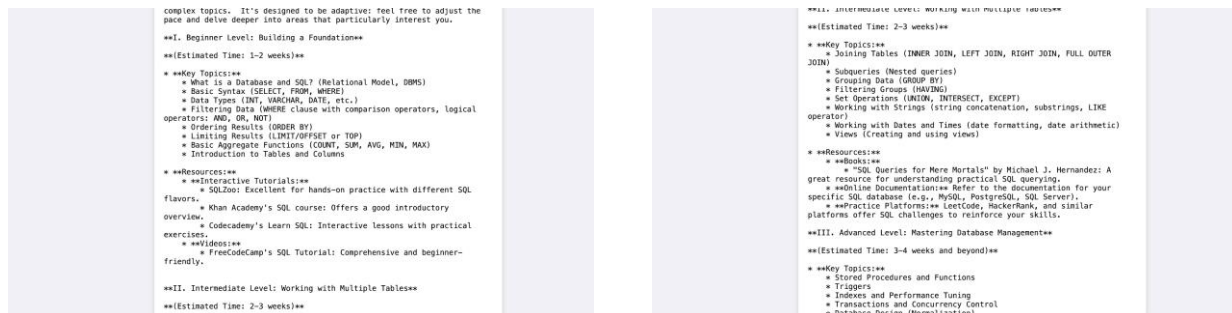
From topic to guided curriculum

The screenshots demonstrate the learning-path capability using SQL as the sample topic.



Learning recommendation request and initial output

Beginner-level SQL guidance



Intermediate-level progression

Advanced-level topics and resources

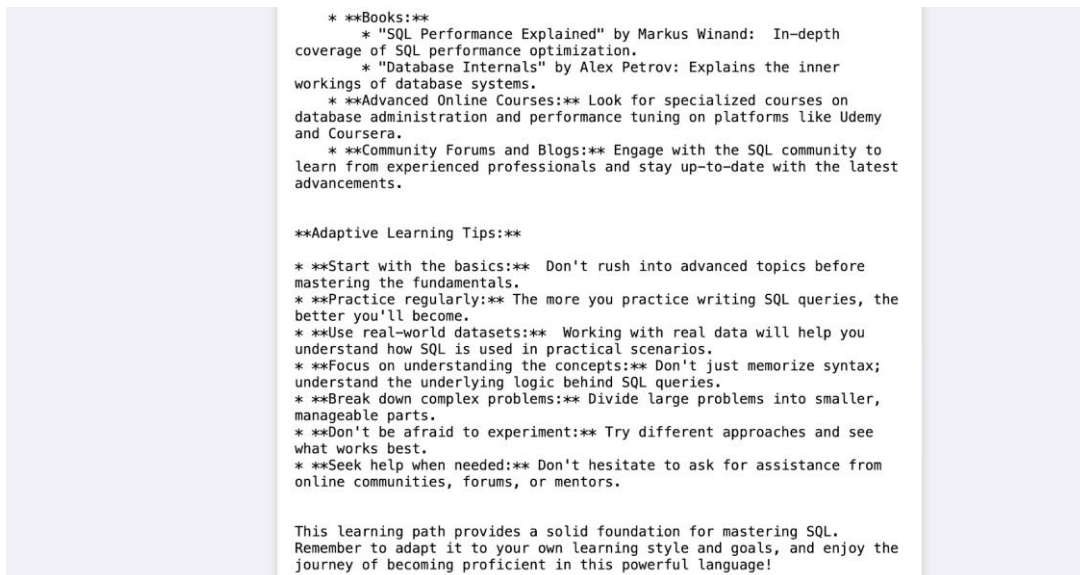


Figure 10. Extended learning-path content with resources and adaptive learning tips.

Observed structure

The demonstrated output progresses through levels, includes estimated learning time, lists key topics, and provides resource suggestions and adaptive learning tips.

11 | DEPLOYMENT & EXECUTION

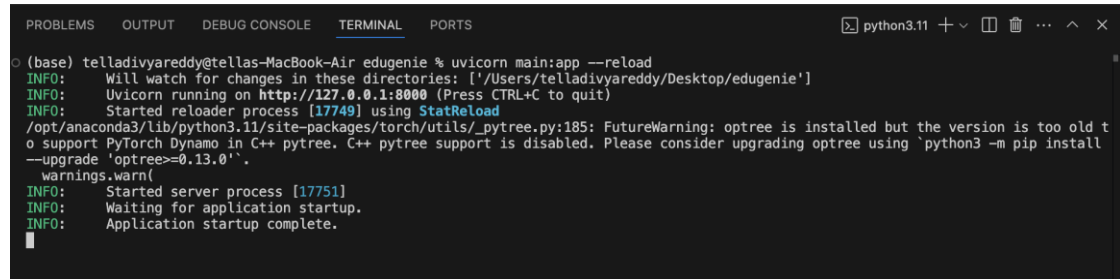
Running EduGenie locally

The project can be started locally through Uvicorn and accessed through the browser.

Local execution

```
uvicorn main:app --reload
```

Application URL: <http://127.0.0.1:8000>

A screenshot of a terminal window with a dark background. The window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. The terminal shows the command 'uvicorn main:app --reload' being executed. The output includes several informational messages: 'Will watch for changes in these directories: [...]', 'Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)', 'Started reloader process [17749] using StatReload', a warning about 'optree' being outdated, 'Started server process [17751]', 'Waiting for application startup.', and 'Application startup complete.'.

```
python3.11 + - + ... ^ x
(base) telladivyaireddy@tellas-MacBook-Air edugenie % uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/Users/telladivyaireddy/Desktop/edugenie']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17749] using StatReload
/opt/anaconda3/lib/python3.11/site-packages/torch/utils/_pytree.py:185: FutureWarning: optree is installed but the version is too old to support PyTorch Dynamo in C++ pytree. C++ pytree support is disabled. Please consider upgrading optree using 'python3 -m pip install --upgrade 'optree>=0.13.0''.
  warnings.warn(
INFO: Started server process [17751]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 11. Uvicorn terminal output used to verify local startup.

Functional test checklist

- ✓ Ask a question
- ✓ Get an explanation
- ✓ Generate a quiz
- ✓ Summarize content
- ✓ Get a personalized learning plan

12 | PROJECT OUTCOMES

What the implementation demonstrates

The completed project combines AI integration, modular backend design, frontend interaction, and practical learner workflows.

01 Modular backend	02 Model strategy	05 Learning modes	05 REST routes
------------------------------	-----------------------------	-----------------------------	--------------------------

Key implementation outcomes

- A FastAPI backend provides dedicated endpoints for each learning capability.
- Gemini is integrated for Q&A, summarization, quiz generation, and learning recommendations.
- LaMini-Flan-T5-783M provides lightweight concept explanation.
- The frontend exposes the capabilities through a straightforward HTML/CSS interface.
- The project demonstrates both direct question answering and longer-form guided learning.

Learning outcomes from development

- Prompt engineering for structured AI responses.
- API integration between a web frontend and AI-enabled backend.
- Handling structured JSON output for quiz generation.
- Designing modular AI functionality around distinct user intents.
- Balancing simplicity, usability, model capability, and lightweight execution.

13 | CHALLENGES & FUTURE SCOPE

From working prototype to broader learning platform

The source document identifies practical challenges and several directions for future enhancement.

Development challenges identified

- Ensuring accuracy in AI-generated responses.
- Maintaining performance across devices.
- Balancing personalization with general usability.
- Improving prompt structures through testing and refinement.
- Addressing data privacy considerations.

Future enhancement roadmap

01	Voice interaction — Hands-free learning through spoken commands and queries.
02	Multilingual support — Broader accessibility across language groups.
03	Mobile application — Learning access anytime, including planned offline capabilities.
04	Progress dashboards — Visualize learning journeys using metrics and insights.
05	Gamification — Badges, streaks, and challenge-based assessments.
06	Adaptive analytics — Guide content based on learner strengths and weaknesses.
07	Collaboration — Group study, teacher/parent dashboards, and LMS integration.
08	Image & PDF input — Snapshot-based doubt solving and resource summarization.
09	Smart notifications — Improve continuity through real-time synchronization and alerts.

14 | CONCLUSION

EduGenie: a learner-first AI foundation

A polished summary of the project's contribution and direction.

EduGenie has been developed as an accessible AI-powered educational assistant that combines cloud-based intelligence with a lightweight web interface. Its modular design supports question answering, concept explanation, quiz generation, summarization, and personalized learning paths.

The implementation demonstrates how generative AI can turn static learning material into a more interactive experience. The separation of specialized modules, REST endpoints, and frontend interaction also provides a practical foundation for future upgrades.

The project documentation identifies accuracy, performance, personalization, prompt refinement, and privacy as important engineering considerations. Future scope includes voice interaction, multilingual support, mobile access, progress tracking, gamification, adaptive learning, collaboration, LMS integration, image/PDF input, and smart notifications.

Project takeaway

EduGenie is positioned not simply as a question-answering tool, but as a foundation for a personalized learning companion that can evolve with the learner.

— END OF PROJECT DOCUMENTATION —

Submitted by

Varshini Loviya C , Sakthi Harish M , Pranesh M , Sri Karthick Kumar G

Mentor: Krishnan N

Date: 23/09/2026